

第二章 应用层

1.应用层协议原理

1. 网络应用的体系结构：

- C/S模式
- P2P模式
- 混合体

2. C/S模式：

- 服务器：一直运行；有固定的IP地址和周知的端口号（约定的）；扩展性：由数据中心进行扩展，扩展性差。
- 客户端：主动与服务器通信；与互联网有间歇性的连接；可能是动态的IP地址；不直接与其他客户端通信。

3. P2P模式：

- 几乎没有一直运行的服务器
- 任意端系统之间可以进行通信
- 每一个节点即是客户端又是服务器

自扩展性：新的peer节点带来新的服务能力，当然也带来新的服务请求

- 参与的主机间歇性连接且可以改变IP地址：这使得难以管理
- 例子：Gnutella，迅雷

4. 混合体：

- Napster：文件搜索是集中式的，向中心服务器请求查询；文件传输是P2P的，在任意主机之间进行。
- 即时通信：在线检测是集中的，当用户上线时，向中心服务器注册IP地址；用户与中心服务器联系，以找到其在线好友的位置。两个用户之间采用P2P聊天。

5. 进程通信：

- 进程：在主机上运行的应用程序
- 在同一个主机内，使用进程间通信机制通信（由操作系统定义）
- 不同主机，通过交换报文来通信
- 客户端进程：发起通信的进程
- 服务器进程：等待连接的进程

6. 分布式进程需要解决的问题：

- 进程标示和寻址问题？
- 传输层——应用层提供服务是如何的？
 - 位置：层间界面的SAP
 - 形式：应用程序接口API
- 如何使用传输层提供的服务？
 - 定义应用层协议：报文格式，解释，时序等

- 编制程序，使用OS提供的API，调用网络基础设施提供通信服务传报文，实现应用时序等

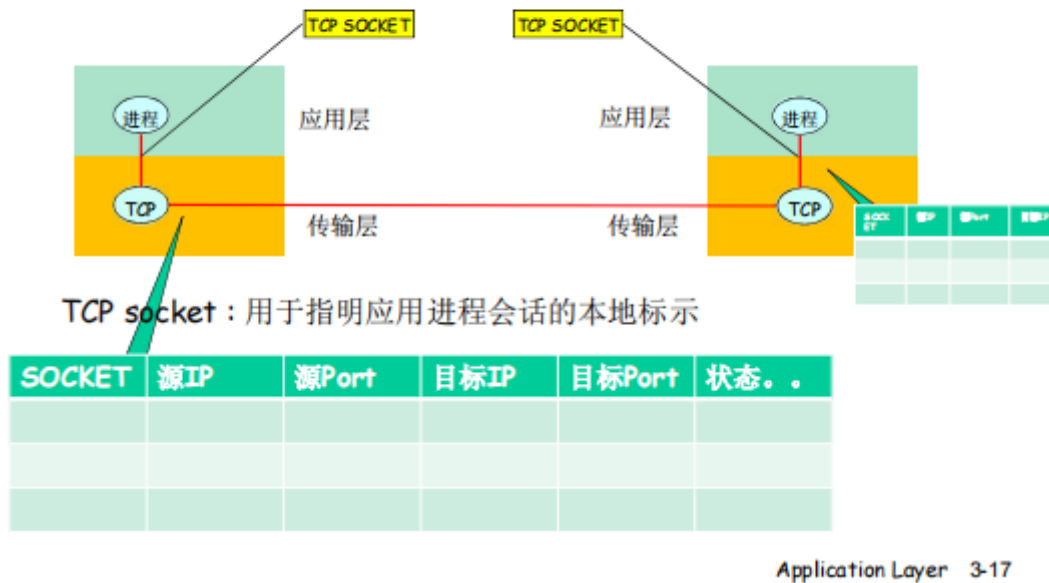
7. 问题1：对进程进行编址

- 进程为了接收报文必须有一个标识，即SAP
- 对于主机有唯一的32位IP地址，但是仅有IP地址不能唯一标识一个进程，因为在一个端系统上有很多应用进程
- 采用的传输层协议：TCP/UDP
- 端口号（Port Numbers）
- 一个进程：用IP+port标示端节点
- 本质上一对主机进程之间的通信由两个端节点构成

8. 问题2：传输层提供的服务——需要串钩层间的信息

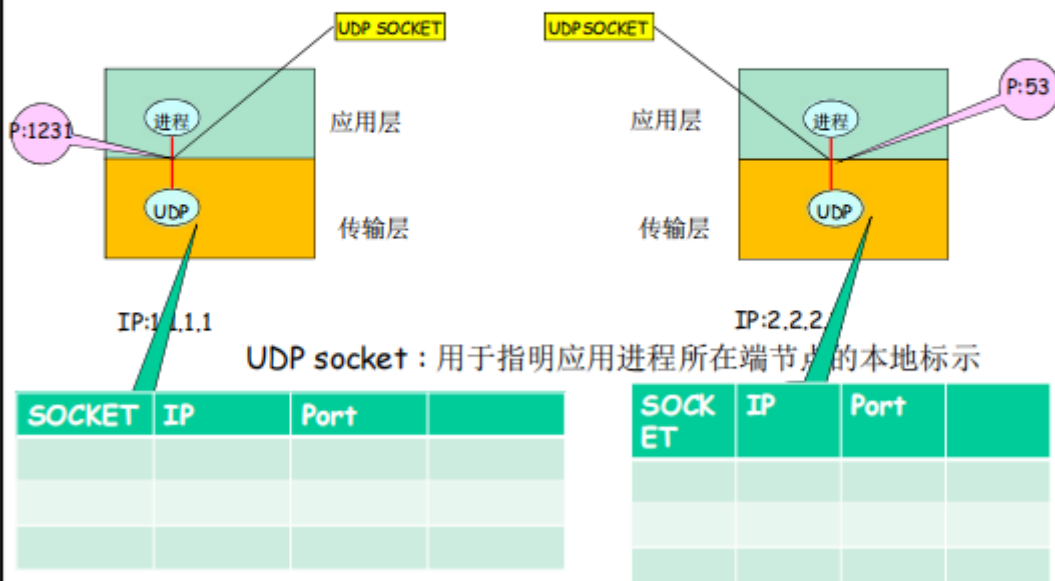
- 层间接口需要携带的信息：
 - 要传输的报文
 - 谁传的：源
 - 传给谁：目标
- 传输层实体根据这些信息进行TCP报文段（UDP数据报）的封装
 - 源端口号，目标端口号，数据等
 - 间IP地址往下交IP实体，用于封装IP数据报：源IP，目标IP
- Socket的引入：考虑到如果每次都传输报文携带这么多的信息很麻烦，使用代号来标识通信的双方或单方，这就是Socket的用途。
- TCP Socket：
 - TCP服务，两个进程之间的通信需要之前建立连接，这个连接会持续一段时间，通信关系稳定。
 - 可以用一个整数表示两个应用实体之间的通信关系，本地标示
 - 使得穿过层间接口的信息量最小
 - TCP socket:源IP，源端口，目标IP，目标端口，一个4元组

TCP socket



- UDP Socket:
 - UDP服务, 两个进程之间的通信需要之前无需建立连接; 每个报文都是独立传输的; 前后报文可能给不同的分布式进程。
 - 因此只能用一个整数表示本应用实体的标示
 - 使得穿过层间接口的信息量最小
 - UDP socket:本IP, 本端口, 2元组
 - 但是传输报文时, 必须要提供对方的IP,port; 接收报文时, 传输层需要上传对方的IP,port

UDP socket



- 进程向套接字发送报文或从套接字接收报文, 就像一个门户一样

9. 问题3: 如何使用传输层提供的服务实现应用

- 应用层协议：定义了运行在不同端系统上的应用进程如何相互交换报文
 - 交换的报文类型：请求报文和应答报文
 - 各种报文的语法：报文中的各个字段及其描述
 - 字段的语义
 - 进程何时、如何发送报文及对报文进行响应的规则
- 应用协议仅仅是应用的一个组成部分
- 如何描述传输层的服务：
 - 数据丢失率
 - 延迟
 - 吞吐量
 - 安全性
- 传输层提供的服务：TCP/UDP
- UDP存在的必要性：
 - 能够区分不同的进程，而IP服务不能
 - 无需建立连接
 - 不做可靠性的工作，适合对实时性要求高而对正确性要求不高的应用
 - 没有拥塞控制和流量控制，能够按照设定的速度发送数据
- 安全TCP：TCP/UDP都没有加密，明文传输
加了SSL，在TCP上面实现，提供加密的TCP连接，SSL在应用层

2.Web和HTTP

2.1 Web

- Web页由一些对象组成
- 对象可以是HTML文件、JPEG图像、Java小程序、声音剪辑文件等
- Web页含有一个基本的HTML文件，该基本HTML文件又包含若干对象的引用
- 通过URL对每个对象进行引用
- URL格式：
协议名：//用户：口令@主机名/路径名/端口

2.2 HTTP

1. HTTP：超文本传输协议，是Web的应用层协议，采用客户/服务器模式。

- 客户：请求、接收和显示Web对象的浏览器
- 服务器：对请求进行响应，发送对象的Web服务器

2. 使用TCP：

- 客户发起一个与服务器的TCP连接（建立套接字），端口号为80
- 服务器接收客户的TCP连接
- 在浏览器（HTTP客户端）与Web服务器（HTTP服务器）交换HTTP报文
- TCP连接关闭

3. HTTP是无状态的：服务器并不维护关于客户的任何信息

2.3 HTTP/1.0

非持久HTTP：

- 最多只有一个对象在TCP连接上发送
- 下载多个对象需要多个TCP连接
- HTTP/1.0使用非持久连接

2.4 HTTP/1.1

持久HTTP：

- 多个对象可以在一个TCP连接上传输
- HTTP/1.1默认使用持久连接

2.5 HTTP/2

HTTP/2的主要目标是减小感知时延，其手段是经单一TCP连接使请求和响应多路复用，提供请求优先次序和服务堆栈，并提供HTTP首部字段的有效压缩，HTTP/2不改变HTTP方法、状态码、URL或首部字段，而是改变数据格式化方法以及客户和服务之间的传输方式。

经单一TCP连接发送一个Web页面中的所有对象存在队首阻塞（HOL）问题。HTTP/1.1采用打开多个并行的TCP连接，从而让同一Web页面上的多个对象并行地发送给浏览器。

HTTP/2的基本目标之一是摆脱传送单一Web页面时的并行TCP连接。

- HTTP/2成帧：将每个报文分成小帧，并且在相同TCP连接上交错发送请求和响应报文。
- 响应报文的优先次序和服务堆栈

2.6 HTTP/3

QUIC是一种新型的传输协议，它在UDP上实现。HTTP/3是一种设计在QUIC之上的新HTTP。

cookies和Web缓存（条件GET方法）

3.FTP*

1. 向远程主机上传文件或从远程主机接收文件
2. ftp:RFC 959,端口号21
3. 有状态
4. ftp命令、响应：

FTP命令、响应

命令样例：

- ❑ 在**控制连接**上以**ASCII**文本方式传送
- ❑ **USER username**
- ❑ **PASS password**
- ❑ **LIST**：请服务器返回远程主机当前目录的文件列表
- ❑ **RETR filename**：从远程主机的当前目录检索文件 (gets)
- ❑ **STOR filename**：向远程主机的当前目录存放文件 (puts)

返回码样例：

- ❑ 状态码和状态信息 (同HTTP)
- ❑ **331 Username OK, password required**
- ❑ **125 data connection already open; transfer starting**
- ❑ **425 Can't open data connection**
- ❑ **452 Error writing file**

4.Email

1. 组成部分：

- 用户代理
 - 邮件服务器
 - 简单邮件传输协议：SMTP
2. 用户代理：又叫邮件阅读器；用来撰写、编辑和阅读邮件；如Outlook
3. 邮件服务器：邮箱中管理和维护发送给用户的邮件
4. SMTP：使用TCP传输，端口号25

- 直接传输：从发送方服务器到接收方服务器
- 传输的三个阶段：握手，传输报文，关闭
- 命令/响应交互：命令：ASCII文本；响应：状态码和状态信息
- 报文必须为7位ASCII码
- 使用持久连接

5. 邮件访问协议：从服务器访问邮件

- POP：邮局访问协议：用户身份确认并下载，无状态
- IMAP：Internet邮件访问协议：更多特性，在服务器上处理存储的报文，有状态
- HTTP：Hotmail, Yahoo!等；方便

5.DNS

1. DNS的必要性：IP地址标识主机、路由器，但是IP地址不好记忆，不便于使用；由DNS负责将IP地址转换成二进制的网络地址。

2. DNS的主要思路：

- 分层的、基于域的命名机制

- 若干分布式的数据库完成名字到IP地址的转换
- 运行在UDP之上端口号为53的应用服务
- 核心的Internet功能，但以应用层协议实现

3. DNS主要目的：

- 实现主机名——IP地址的转换
- 其他：
 - 主机别名到规范名字的转换 (host aliasing)
 - 邮件服务器别名到邮件服务器正规名字的转换
 - 负载均衡 (load distribution)

4. DNS域名结构、层次树状结构：

- Internet根被分为几百个顶级域 (top)；又分为通用的和国家的
- 每个域下再划分子域，树叶是主机
- 共有13个根名字服务器
- 5. 域名的管理：一个域管理其下的子域；域与物理网络无关，域的划分是逻辑的，而不是物理的。一个域的主机可以不在一个网络，一个网络的主机可以不在一个域。
- 6. 区域：将DNS名字空间划分为互不相交的区域
- 7. 名字服务器：每个区域都有一个名字服务器 (Name Server)，维护着它所管辖区域的权威信息。名字服务器允许被放置在区域之外，以保障可靠性。
- 8. 权威DNS服务器：组织机构的DNS服务器，提供组织机构服务器可访问的主机和IP之间的映射。组织机构可以选择实现自己维护或由某个服务器提供商维护。
- 9. TLD服务器 (顶级域)：负责顶级域名和所有国家级的顶级域名。如Network solutions公司维护的com TLD Server.

10. 区域名字服务器维护资源记录 (RR)：

资源记录：

- 作用：维护域名——IP地址的映射关系
- 位置：Name Server的分布式数据库中

RR格式：(domain name,ttl,type,class,value)

TTL:生存时间，决定了资源记录应当从缓存中删除的时间。

11. DNS大致工作过程：

- 应用调用解析器
- 解析器作为客户向Name Server发出查询报文 (封装在UDP段中)
- Name Server返回响应报文

12. 本地名字服务器 (Local Name Server)：并不严格属于层次结构。每个ISP都有一个本地的DNS服务器，也叫默认名字服务器。当一个主机发起一个DNS查询时，被送到本地DNS服务器，再转发到层次结构中，起到相当于一个代理的作用。

13. 两种查询方式：

- 递归查询：根服务器负担太重
- 迭代查询：根 (及各级域名) 服务器返回的不是查询结果，而是下一个DNS地址，最后由权威名字服务器给出解析结果

14. 为提高性能的方法：缓存

- 一旦名字服务器学到了一个映射，就将该映射缓存起来
- 根服务器通常都在本地服务器中缓存，缓存使得根服务器不用经常被访问，提高效率
- 可能的问题：如果情况变化（映射关系改变），缓存结果和权威资源记录不一致
- 解决方法：TTL

15. 维护问题：新增一个域

16. 攻击DNS：总的来说，DNS比较健壮

6.P2P

6.1 P2P架构

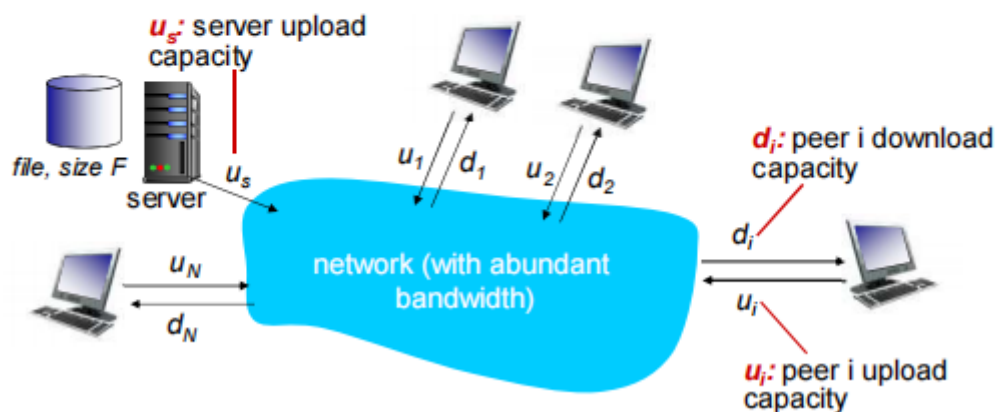
- 没有（或极少）一直运行的服务器
- 任意端系统都可以直接通信
- 利用peer的服务能力
- peer节点间歇上网，每次IP地址都有可能变化
- 例子：文件分发（BitTorrent）、流媒体、VoIP(Skype)

6.2 P2P和C/S的比较

文件分发：C/S vs P2P

问题：从一台服务器分发文件（大小 F ）到 N 个peer需要多少时间？

○ Peer节点上下载能力是有限的资源



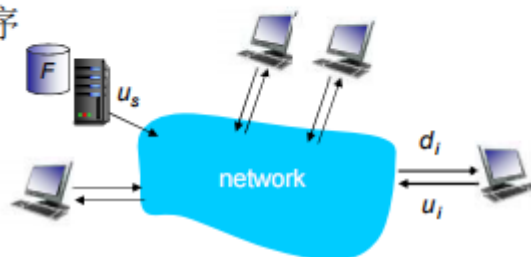
文件分发时间：C/S模式

- **服务器传输：** 都是由服务器发送给peer，服务器必须顺序传输（上载）N个文件拷贝：

- 发送一个copy: F/u_s
- 发送N个copy: NF/u_s

- ❖ **客户端：** 每个客户端必须下载一个文件拷贝

- $d_{\min}$ = 客户端最小的下载速率
- 下载带宽最小的客户端下载的时间: $F/d_{\min}$



采用C-S方法
将一个F大小的文件
分发给N个客户端耗时

$$D_{c-s} \geq \max\{NF/u_s, F/d_{\min}\}$$

随着N线性增长

文件分发时间：P2P模式

- **服务器传输：** 最少需要上载一份拷贝

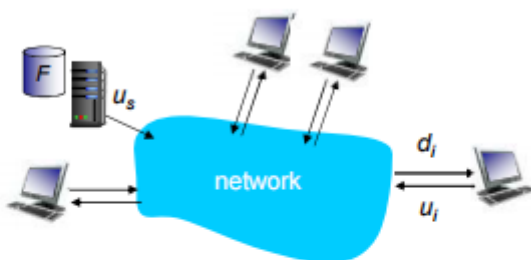
- 发送一个拷贝的时间: F/u_s

- ❖ **客户端：** 每个客户端必须下载一个拷贝

- 最小下载带宽客户单耗时: $F/d_{\min}$

- ❖ **客户端：** 所有客户端总体下载量NF

- 最大上载带宽是: $u_s + \sum u_i$
- 除了服务器可以上载，其他所有的peer节点都可以上载



采用P2P方法
将一个F大小的文件
分发给N个客户端耗时

$$D_{P2P} \geq \max\{F/u_s, F/d_{\min}, NF/(u_s + \sum u_i)\}$$

分子随着N线性变化，每个节点需要下载，整体下载量随着N增大 ...

... 分母也是如此，随着peer节点的增多
每个peer也带了服务能力

Application Layer 2-104

6.3 P2P文件共享

1. P2P文件共享问题：

- 如何定位所需的资源
- 如何处理对等方的加入和离开

2. 可能的方案：

- 集中
- 分散

- 半分散

6.4 集中式目录

以*Napster*为例，当对等体连接时，告诉中心服务器它的IP和内容

存在的问题：

- 单点故障，如果中心服务器宕机，则整个网络都会崩溃
- 性能瓶颈
- 侵犯版权

6.5 查询泛洪

以*Gnutella*为例，没有中心服务器，采用开放式共享协议，类似于广播，范围有限，发出请求后，能响应的服务器回应。

1. 覆盖网络：

- 如果对等方X和Y维护了一条TCP协议，则说X和Y之间有一条边
- 所有活跃的对等方和边组成覆盖网络
- 边不是物理通信链路
- 给定对等方连接的覆盖网络路径中的节点数少于10个，即TTL小于10

2. 加入对等方：

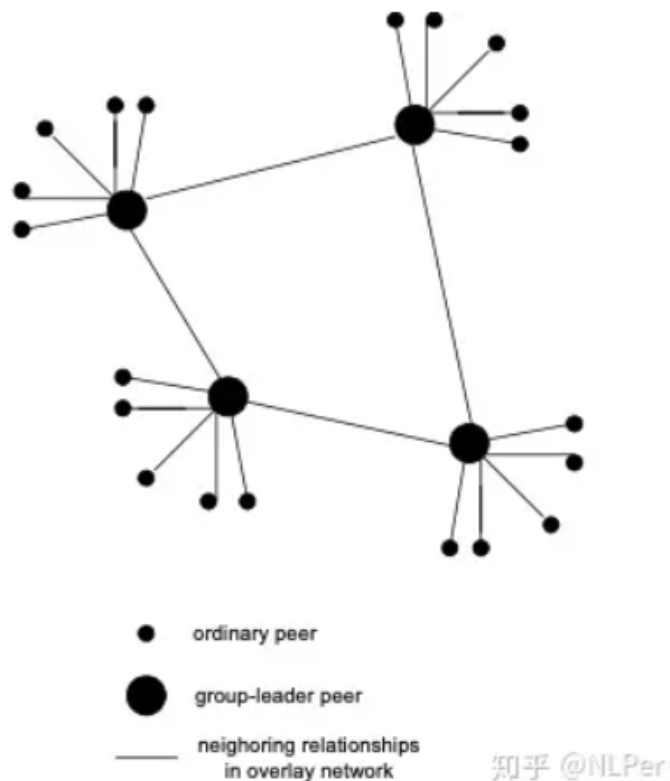
- 加入对等方X必须发现在*Gnutella*网络中的其他对等方:使用对等方列表
- X试图与该列表上的对等方建立一条TCP连接，直到与Y创建一条连接
- 向Y发送一个Ping报文;Y转发该Ping报文
- 所有的对等方接收Ping报文并响应一个Pong报文
- X接收到许多Pong报文，然后能同某些其他对等方建立TCP连接

3. 对等方离开：

- 主动离开:离开接点的所有对等方都会刷新自身的激活对等方列表,并开始与列表中的新的对等方建立连接
- 断网:发送信息的时候对等方没有响应,则表明对等方离开,节点刷新自身的激活对等方列表,并开始与列表中的新的对等方建立连接

6.6 KaZaA

1. 纯P2P的改进，超级节点技术



- 每个对等方要不被指派 为组长，要不被指派给一个组长
- 对等方和组长之间建立 TCP连接
- 组长之间建立TCP连接
- 组长维护它的子对等方 共享的内容

2. 特点:

- 请求排队：限制对等方并行上载数量，新的请求进行排队
- 激励优先权：根据不同的上载下载比例优先服务贡献大者
- 并行下载：将一个文件分成若干段，从多个对等方并行下载

6.7 P2P文件分发：BitTorrent

参与一个特定文件分发的所有对等方的集合称为一个洪流，一个洪流中的对等方彼此下载等长度的文件块。

追踪器Tracker：跟踪参与洪流的所有对等方。

一报还一报。

7.CDN

1. DASH：基于HTTP的动态自适应流化服务

- 服务器：
 - 将视频文件分割成多个块
 - 每个块独立存储，编码于不同码率

- 告示文件：提供不同块的URL
- 客户端：
 - 先获取告示文件
 - 周期性地测量服务器到客户端的带宽
 - 查询告示文件，在一个时刻请求一个块，HTTP头部指定字节范围
- 2. CDN：全网部署缓存节点，存储服务内容，就近为用户提供服务，提高用户体验
- 3. 两种形式：
 - enter deep深入：将CDN服务器深入到许多接入网中
 - bring home邀请做客：部署在少数关键位置，如将服务器簇安装于POP附近
- 4. 用户从CDN中请求内容，重定向到最近的拷贝，请求内容；如果网络路径拥塞，可能选择不同的拷贝